

智能数据湖计算 AI DLC

AI Native 数据湖的 Spark + Ray 一体化实践

01

—
趋势与洞察

02

—
腾讯云 AI DLC

03

—
产品演示

04

—
总结与规划

01

趋势与洞察

智能计算范式的演进 | 数据 · 计算 · 训练



每个计算时代都有专属的计算框架与引擎；每次范式更替，都在淘汰旧架构、兴起新计算



核心洞察



产业观察 | 大数据 × AI 的双向奔赴

过去是大数据平台里「加 AI 功能点」；这一次，是两个阵营带着各自的解决方案，认真地走向彼此



我们的判断

大数据与 AI 各自擅长、天然互补 —— 当两个阵营站到同一张桌前，谁能把「融合」做成产品，谁就定义下一代数据平台

数据工程方式的转变 | 开发 · 分析 · 服务



确定性系统 一次写对

逻辑一次写进代码，测试通过即永远正确；追求的是更低的缺陷率，迭代越少越稳。

AI 系统 迭代速度

输出概率化、数据会漂移，不可能一次写对；迭代速度 = 感知-判断-修复的端到端延迟 = 竞争力本身。

开发 *Development*

传统 SQL / ETL 声明式，关系代数计算，产出机构化的表



AI Python-first + 异构计算 pipeline，产出可训练语料 和 Agent 轨迹、记忆与上下文

加工 Agent 上下文

分析 *Analysis*

传统 BI 报表 / OLAP，回答「发生了什么」



AI 特征工程 + 后训练定制，让模型学会分析并驱动 Agent 决策

教 Agent 判断

服务 *Service*

传统 数据服务 · API 取数，一次上线即稳定



AI 模型即服务 MaaS · Agent 运行时，闭环进化

让 Agent “上岗”

开发、分析、服务，正在合成一条 Agent 持续进化的链路——数据、模型、算力每个维度都在快速迭代，而这条链路的运转速度，决定了 AI 的业务效果；数据平台的演进方向，就是让这条链路转得更快

AI 时代，需要什么样的数据平台



智能计算范式的演进（数据·计算·训练） + 数据工程方式的转变（开发·分析·服务） → 新平台的设计思路

AI 时代，必然需要属于自己的计算框架与计算引擎

01

支持异构

Heterogeneous-Native

硬件从 CPU 到 CPU + GPU，数据从结构化到多模态；集成异构计算框架

02

AI 原生开发

AI-Native by Design

以 Python-first 与分布式框架为原生底座，保持开放生态，让开发、分析、服务回归 AI 工程师的习惯与效率

03

数据算力同底座

One Unified Foundation

训练、分析、推理共享同一份数据，闭环就地自转

04

面向 Agent 进化

Built for Agent Evolution

Agent 轨迹与反馈就地沉淀回流，平台服务于「链路运转效率」，而非一次性的任务执行

循着这四条设计思路，我们把它落成了一个原生的 AI 计算底座

02 + 03

智能数据湖计算 AI DLC

腾讯云数据湖计算 DLC → 智能数据湖计算 AI DLC



从大数据计算平台，进化为原生支撑 Agent 运行并自我进化的智能底座



接下来 我们逐一展开这四项产品升级 —— 计算范式 · 引擎性能 · 元数据 · Agentic 能力

02-1

支持异构

AI 时代的资源天生异构 —— 平台在计算与存储两端，把异构算力与多模态数据统一纳管、按需调度

计算资源 异构算力池

统一纳管多元算力，细粒度动态调度到任意负载

1 多元异构算力

CPU / GPU 统一纳入同一资源池，屏蔽硬件差异

2 细粒度动态调度

同一份算力同时跑数据处理、后训练与在线推理，按需分配

3 高速互联加速

RDMA 等加速设施，提速 GPU 间上下文交换与集合通信

存储资源 异构·高性能存储

按 AI 全生命周期组织存储

1 训练·高吞吐加载

海量多模态数据 + TB/s 带宽加载权重与 Checkpoint

2 推理·低时延状态

KVCache / Embedding 高速流转，毫秒级命中支撑高并发推理

3 Agent·就地沉淀回流

轨迹 / 记忆 / 产物就地留存，可检索、可回流为下一轮语料

资源层的使命 把异构的算力与异构的数据，变成一个可统一调度、按需伸缩的资源底座 —— 让上层引擎无需关心硬件与格式差异。

保护开发者既有习惯，不为资源/运维操心；自研内核在性能与稳定性上大幅领先开源

资源层 · 统一容器化资源调度

异构算力池 · 统一调度 · 自动扩缩 · 滚动升级 · 故障自愈 —— 引擎任务由调度层按需拉起 Pod，按时长 / 资源计费

DLC 数据计算

100% 兼容 Spark · 自研内核全面升级

数据并行 · 结构化与半结构化处理

数据加工与数仓建设

大规模表 join、聚合、清洗，数据并行处理

特征工程与数据准备

为下游训练准备特征与样本数据

大规模机器学习

Spark MLlib 跑经典 ML 算法与统计建模

BI 分析与报表

结构化数据的即席查询与分析洞察

▲ 性能领先开源

自研向量化引擎：1TB TPC-DS 性能 3.6X、CPU 占用 ↓50%

DLC AI 计算

100% 兼容开源 Ray · AI 计算引擎

任务并行 · 异构算力 · AI 原生

多模态数据处理

视频 / 图像 / 文本，CPU+GPU 异构管线

大模型后训练与推理

SFT / RLHF / RL、在线推理服务

Agent 运行时

多步推理、工具调用、长期记忆与协作

强化学习与超参搜索

RL、复杂计算任务、HPC

▲ 性能领先开源

内核优化 + 平台增强：开源 API 零改动，性能大幅领先

双引擎如何配合 Spark 数据准备 → 统一元数据与存储 + 内存格式互换 → Ray 计算，闭环无缝

● Serverless 化计费

按任务 / 时长计费，免集群运维、免资源预留

● 数据不跨系统

Spark 与 Ray 共享同一份资源池与数据底座

● 统一调度与治理

数据与 AI 计算任务统一管控、统一权限与配额

● 统一可观测

日志 / 指标 / 追踪打通，无需跨系统排查



运维与可观测

Logs
作业 / 系统 / 应用日志关联检索

Monitor
节点健康、CPU / GPU / 内存、吞吐 / QPS

Alerts
阈值 / 趋势 / 关键智能告警

Profiling
算子耗时、GPU Kernel、内存 / 显存占用

Tracing
全链路追踪 · 训练链路白盒化

TCRay 价值 开源 Ray 的全栈能力 + 全托管的工程化体验：内核优化、平台能力、可观测一应俱全，让团队专注业务而非基础设施。

02-2

AI 原生开发

基于 Ray 框架构建 | 全新 SQL-First AI Function | 50+ 多模态 / LLM / LM 算子

性能亮点

Performance Highlights

3x

推理吞吐提升

相比开源 Data-Juicer，推理吞吐实现 3 倍提升

100%

GPU 资源利用率

GPU 得到充分释放，利用率长期稳定接近 100%

Best

CPU/GPU 协同调度

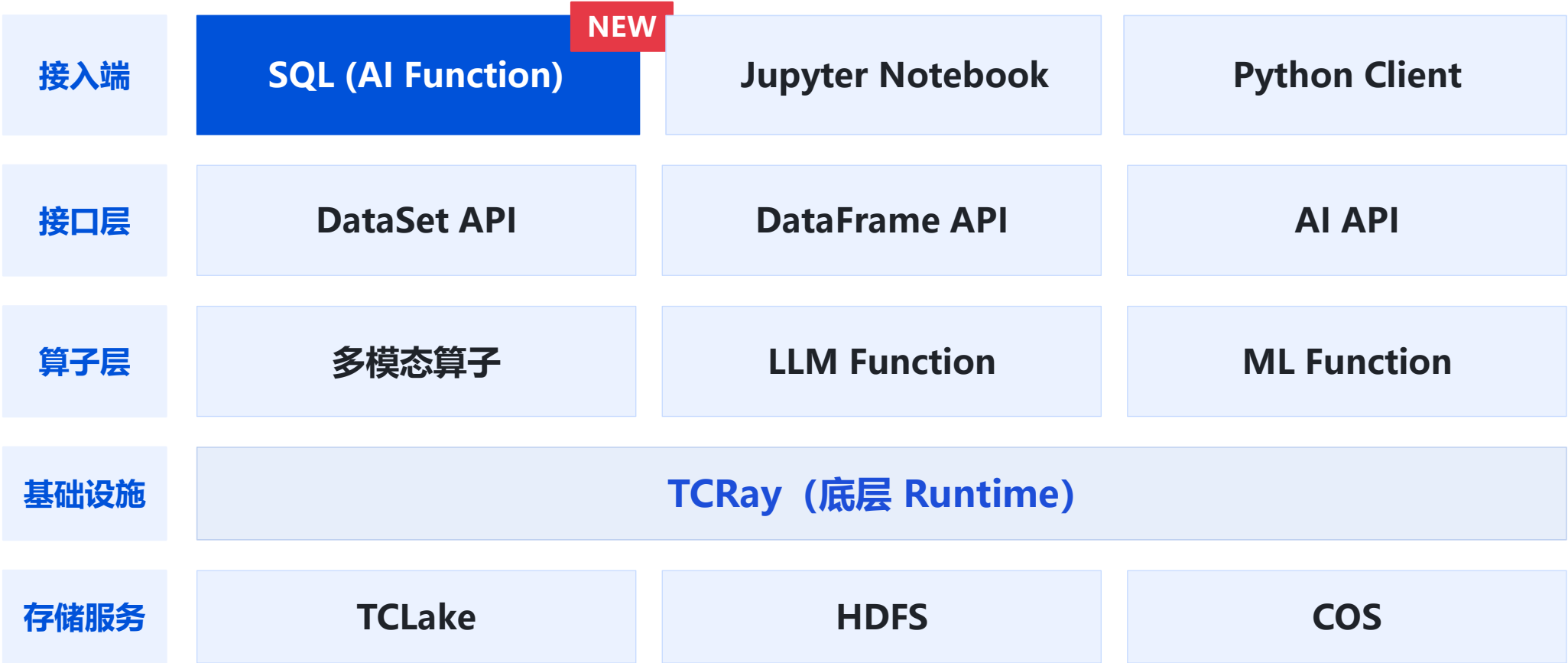
Xpark 智能调度 (Zero-Copy) + Ray Data 任务并行

Xpark ≈ 100%

Spark ≈ 50%

5 层架构

Architecture



DLC 多模态计算引擎 (Xpark) — 基于 Ray 框架构建，为数据开发者屏蔽分布式复杂度；AI 原生开发体验，让 SQL 和 Python 直接编排多模态与 LLM 流水线，无需关心底层集群。

1TB TPC-DS 查询分析场景性能提升 3.6X — 腾讯云增速最快的自研计算引擎，100% 兼容开源 Spark、原生支持增量计算

3.6X

TPC-DS 1TB 查询分析性能

1TB TPC-DS 标准数据集，端到端加速

5X

计算密集型查询最高加速

q23 / q24 / q64 / q78 等典型重计算

↓ 50%

CPU 使用率下降

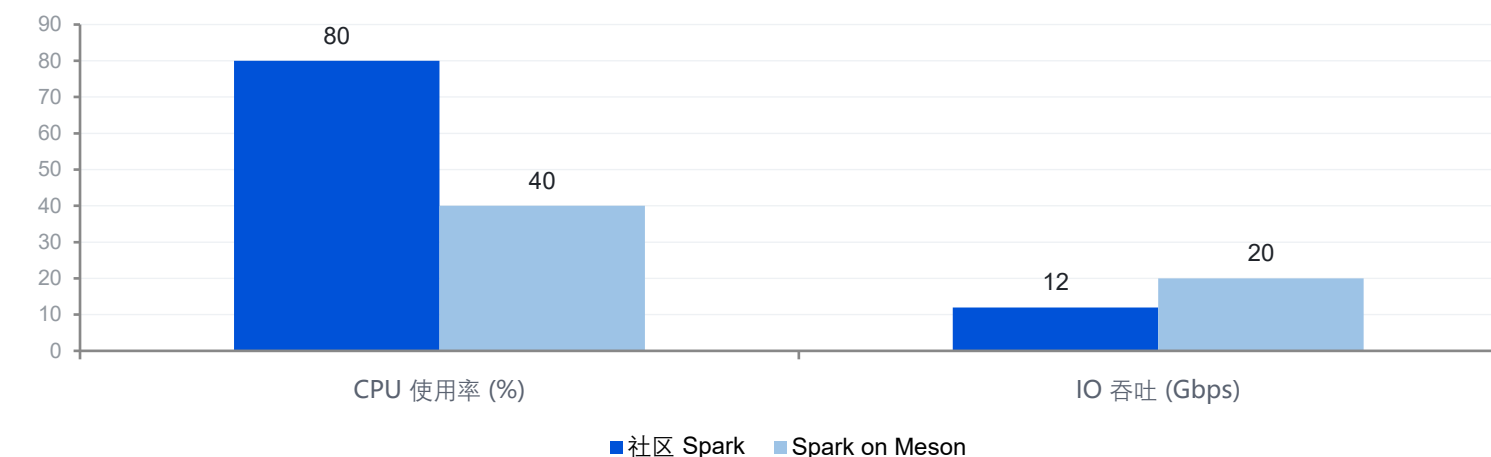
由 80% 降至 40%，同等负载算力更省

100+

广泛服务腾讯云客户

已稳定支撑 100+ 客户的生产作业

CPU 使用率 & IO 吞吐



+

原生增量计算

数据 / 模型 / 中间结果就地增量，避免全量重算；配套动态表、配套物化视图等产品能力

✓

完全兼容 Spark

API、SQL 语法、UDF、生态组件 100% 兼容；存量作业零改动迁入，开源生态持续受益

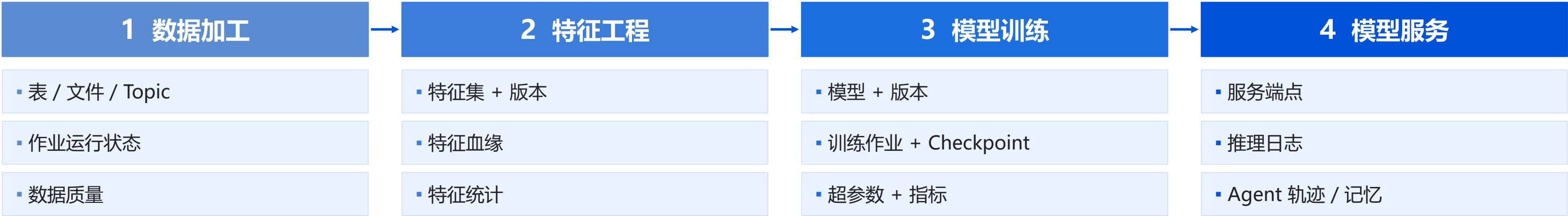
性能数据来源：（1TB TPC-DS，社区 Spark vs Spark on Meson） — 生产环境实际收益以客户 workload 为准。

02-3

数据算力同底座 – 数据 & 元数据

纳管数据对象与 AI 对象，从数据加工到模型服务全流程的治理与血缘

全流程血缘：从数据加工到模型服务，每一步的元数据与运行状态都纳管



01
统一纳管

Unified Catalog

数据对象（表 / 文件 / Topic）+ AI 对象（模型 / 特征 / Agent 轨迹），一套模型统一管理。

02
直接注册

Direct Registration

引擎主动注册元数据与运行状态，非被动采集；变更即时双向同步。

03
端到端治理

End-to-End Governance

访问控制、审计、发现、血缘追踪，从数据到模型全链路可追溯。

04
多引擎接入

Multi-Engine

Spark / Ray / Xpark / Meson 等数据与 AI 引擎统一接入，运行状态自动归集。

AI 与数据计算 都将自己的数据、作业、模型的运行状态通过 TCLake 统一纳管，实现全流程管控。

02-4

面向 Agent 进化

Open Engine 让客户 Agent 持续进化 + MCP / Skills / SDK 让平台被外部 Agent 调用 + TCInsight 智能自治

Open Engine · 让客户 Agent 跑得更聪明

Open Engine · 客户 Agent 持续进化闭环



▸ Agent 越用越聪明 持续进化

MCP / Skills / SDK · 让平台被任意 Agent 调用

MCP / Skills / SDK / Open API

MCP Server
模型上下文协议

Skills
技能原子化

SDK / Open API
完整编程接入

外部 Agent 调用: WorkBuddy · Codebuddy · Claude Code 等

TCInsight · 大数据智能管家

智能自治 (自感知 / 自诊断 / 自处理 / 自复盘)

自主调优 Agent

自主运维 Agent

预测治理 Agent

模型库

场景库

策略库

数据特征库

多维多层库

▸ 资源降低 15% ▸ 根因定位 4.5h → 30min ▸ VLDB 2025 收录

全链路系统智能：每个阶段都有 Agent 协助用户

1 购买选型 资源规划 Agent

2 迁移上云 智能迁移 Agent

3 日常使用 智能管控 Agent

4 运维调优 智能自治 Agent

AI DLC 功能 7: Open Engine

新建计算引擎

配置目录

1 基本信息

2 计算资源

3 集群镜像

4 存储配置



SQL Warehouse
Spark SQL · 交互式分析



Ray
训练 · 推理 · 分布式 Python



Open Engine Beta
MLflow · Agent



1 基本信息

选择开放引擎 · 名称 · 启动参数 · 描述

选择要创建的开放引擎



MLflow
实验跟踪 · 实验元数据 + Artifact 存储 + 模型注册中心



Agent
Agent 服务托管 · 部署托管你的 Agent 应用，内置可观测与评测集成



引擎名称 *

agent-20260724

描述

一句话描述这个开放引擎的用途

启动参数

可选, 点击「展开」以代码框形式填写启动参数 ▶

2 计算资源

资源包 · 资源组

资源包 / 资源组 *

pkg-prod-cu-yearly · CU·包年 / team-data ▼

team-data

包年 · 余 96 CU

3 集群镜像

Agent 服务运行镜像 (预置主流框架, 内置 OTLP 轨迹上报)

AI DLC 功能 7：基于 Trace 数据发起后训练任务 · 底层执行 Ray Train · 训练完成自动入库



后训练任务

发起后训练

刷新

任务总数

4

训练中

1

已完成

1

失败

1

任务列表

发起后训练（Concept）

每个后训练任务底层都是一个 Ray Train Job · 可在「AI 计算中心 · 模型训练」中查看实时进度

任务 ID	Agent	方法	基础模型	数据集	状态	进度	资源	准确率	提交时间	操作
pt-001	sql-bot	DPO 偏好优化	Qwen2.5-72B v1	近 30 天失败 + 低分 Trace（328 条） 328 样本	已完成	3/3	4 GU × 4	91.3%	2026-06-22 09:00	Ray Train Job ID
pt-002	rag-helper	SFT 监督微调	Qwen2.5-7B v1	产品 RAG 问答标注集（1240 条） 1,240 样本	训练中	2/3	2 GU × 2	—	2026-06-25 06:00	Ray Train Job ID
pt-003	fin-helper	SFT 监督微调	Llama-3.1-70B v2	财务领域监督微调集（3200 条） 3,200 样本	已失败	1/4	8 GU × 4	—	2026-06-20 14:00	Ray Train Job ID
pt-004	sql-bot		Qwen2.5-72B-sql-bot-dpo-v3 v3	在线交互奖励（点赞/点踩，2800 条） 2,800 样本	排队中	—	4 GU × 8	—	2026-06-25 08:00	Ray Train Job ID



已完成 1m49s >

优化建议报告已生成 📄 `slow_task_optimization.html`

最有价值的发现（已实证）： 那条 16.5s 的慢查询，我把同一条 SQL 在热引擎上重跑了一遍——

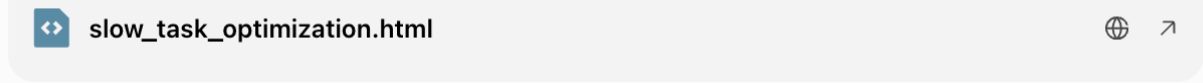
	冷引擎(首次)	热引擎(重跑)
总耗时	16.5s	7.1s
实际计算	8.6s	1.7s
排队	4.5s	1.6s

→ 约 80% 的慢是引擎冷启动（`executor` 缩到 0 后重新拉起）造成的，数据量本身不是瓶颈。 `orsers` 是无分区 Iceberg 单文件表，范围过滤也做不了裁剪。

5 条优化建议（按优先级）：

- 保持引擎温热 —— 收益最高、零改造：让 testly01 这类 StandardSpark 引擎常驻/最小保活，交互查询统一走 State=2 的热引擎，别每次换新引擎。
- 别用 SuperSQL 3.5 引擎跑交互 SQL —— `lcl-35-test1 / lcl-test-cq-35` 会排队 17 分钟后被取消； `spark-35-jar / public-engine` (Presto) 当前也跑不了。
- 高频 COUNT 做物化汇总表 —— 用 DML（连接器已验证可用）把聚合结果落小表，避免每次全表扫。
- 表变大时再建明确 schema + 分区/排序 —— 当前 4.9 万行收益有限；且建表 DDL 在连接器下被拦截，需去 DLC 控制台。
- 排查流式任务 `cc99b84f` 空转 —— 跑 102 分钟、输入 1010 条却输出 0 条、 `idle`，更像逻辑/配置 bug 而非性能问题，建议看 Spark UI 确认 source/sink。

需要我直接帮你做其中哪一项吗？比如把那份 COUNT 查询物化成汇总表，或者去拉 `cc99b84f` 的流式作业日志确认是不是真空转。



查看所有产物 (2) > 查看所有变更 (1) >

📁 📄 🔗 🔊 🔄 🔗 ... 🌐 Hy3

今天帮你做些什么？ @ 引用对话文件， / 调用技能与指令

AI DLC 功能 8：使用 Agent 工具 WorkBuddy 操作 AI DLC

DLC 慢任务优化建议

地域 ap-chongqing · 针对已识别慢任务 · 含一次「冷 vs 热引擎」实证对比

🔗 实证：同一条 SQL，冷引擎 vs 热引擎

结论先行：这条 16.5s 的慢查询，约 80% 是引擎冷启动开销。同一条 `SELECT COUNT(*) FROM samuel1.orsers WHERE _c0 <= 'o_00001010'` 在热引擎（testly01）上仅需 7.1s，实际计算仅 1.7s。

指标	冷引擎 524f7506（首次）	热引擎 4b387114（重跑）	提升
总耗时 TotalTime	16508 ms (16.5s)	7095 ms (7.1s)	↓ 57%
计算耗时 UsedTime	8630 ms (8.6s)	1678 ms (1.7s)	↓ 81%
排队 QueueTime	4494 ms (4.5s)	1555 ms (1.6s)	↓ 65%

注：两次 WHERE 命中行数不同（首次 1000 行，重跑 0 行，因早前 DELETE 已删掉这批），但两者都是对全表 4.9 万行的扫描，计时口径可比。重跑证明了数据量不是瓶颈，引擎状态才是。

📋 优化建议清单

1 保持引擎温热（收益最高、零改造）

现象 / 根因：原始 16.5s 中有约 13s（计算 8.6s + 排队 4.5s）是引擎冷启动 / 弹性伸缩到 0 后重新拉起的开销；同一条 SQL 在热引擎上只需 7.1s，计算仅 1.7s。

建议：把 testly01 这类 StandardSpark 引擎设为常驻 / 最小保活（不让 executor 缩到 0），或在跑关键查询前先发一条轻量 SQL 预热；交互式分析统一走一个 State=2 的热引擎，不要每次换新引擎。

2 不要拿 SuperSQL 3.5 引擎跑交互 SQL

现象 / 根因：之前有两条 SQL 在 `lcl-35-test1 / lcl-test-cq-35` 上排队约 17 分钟后被取消（state=4，超时取消）——这类引擎对交互查询会无限排队最终取消。

建议：交互查询只用健康运行的 StandardSpark 引擎（如 testly01）；`lcl-35-test1 / lcl-test-cq-35 / spark-35-jar / public-engine`(Presto) 当前都跑不了交互 SQL（或被排队取消、或连接器任务类型不兼容）。

3 为高频范围 COUNT 做物化聚合

现象 / 根因： `COUNT(*) ... WHERE _c0<='o_00001010'` 每次都要全表扫 4.9 万行（单文件无裁剪）。如果这类统计被反复调用，每次都重复扫全表。

建议：把聚合结果落进一张小汇总表，按需刷新（DLC 连接器允许 DML：INSERT/DELETE 已验证可用）。例如定时 INSERT 覆盖一张 `daily_count` 表，查询直接读汇总，省掉全表扫。

4 给表建明确 schema + 考虑分区/排序（表较大时再做）

现象 / 根因：`orsers` 是 ICEBERG 无分区表，列名仍是默认 `_c0~_c5`，`_c0` 是 string。无分区 → 范围过滤无法裁剪；列语义不透明 → 无法做时间分区或强类型下推。

建议：若表会增长到千万级或查询以时间区间为主：① 在 DLC 控制台重建表、显式定义列类型（`order_id STRING / order_time TIMESTAMP` 等）；② 按 `order_time` 分区，或按 `_c0` 排序+compact 成少量文件，利用 Iceberg manifest 的 min/max 做文件级跳过。当前 4.9 万行量级做分区收益有限，优先级低。注：建表(DDL)在 MCP 连接器下被拦截，需去 DLC Web 控制台执行。

5 排查流式任务空转（cc99b84f）

现象 / 根因：该 Spark Streaming 作业已跑约 102 分钟仍运行中，但输入 1010 条、输出 0 条、`CmdArgs=idle`——不是性能调优问题，更像逻辑/配置 bug（源头无数据或 sink 未写盘）。

建议：打开报告里的 Spark UI 看 streaming 标签页的 batch 进度；确认 source 是否真有数据流入、sink 写 COS/表的配置是否正确；若确属空转，停掉以免持续占用资源。

把 orsers 全量导出来，别一次拉爆。



已完成 20m35s >

The full export of samuel1.orsers is now complete and verified. Here's the summary:

Export finished — /Users/ming/WorkBuddy/dlc/orsers_export.csv

Check	Result
Data rows	49,990 (exact match to table metadata)
Total lines (incl. header)	49,991
File size	2,873,075 bytes (~2.7 MB)
Columns	_c0 – _c5 (order_id, user_id, product_id, qty, amount, order_time), all 6 present every row
Empty cells	0 across all columns
_c0 uniqueness	49,990 distinct → 0 duplicates
First row	o_00000011, u_000906, p_0058, 2, 275.14, 2025-04-17 15:30:46
Last row	o_00050000, u_000529, p_0009, 3, 2564.67, 2025-10-25 16:46

今天帮你做些什么？ @ 引用对话文件， / 调用技能与指令

默认权限



Hy3



内容由 AI 生成，请核实重要信息

AI DLC 功能 8：使用 Agent 工具 WorkBuddy 操作 AI DLC

orsers 订单数据看板

数据源：DLC · ap-chongqing · samuel1.orsers | 全量导出快照

时间跨度 2024-12-18 ~ 2025-12-18

记录数 49,990

生成 2026/7/24

总交易额 GMV

¥1.44亿

日均约 ¥39.3万

订单总数

49,990

客单价 ¥2,871.21

活跃用户

1,000

人均消费 ¥143,532

商品数

100

累计销量 149,478 件

平均金额

¥2,871.21

中位数金额

¥2,081.92

P90 金额

¥6,605.25

最小 / 最大

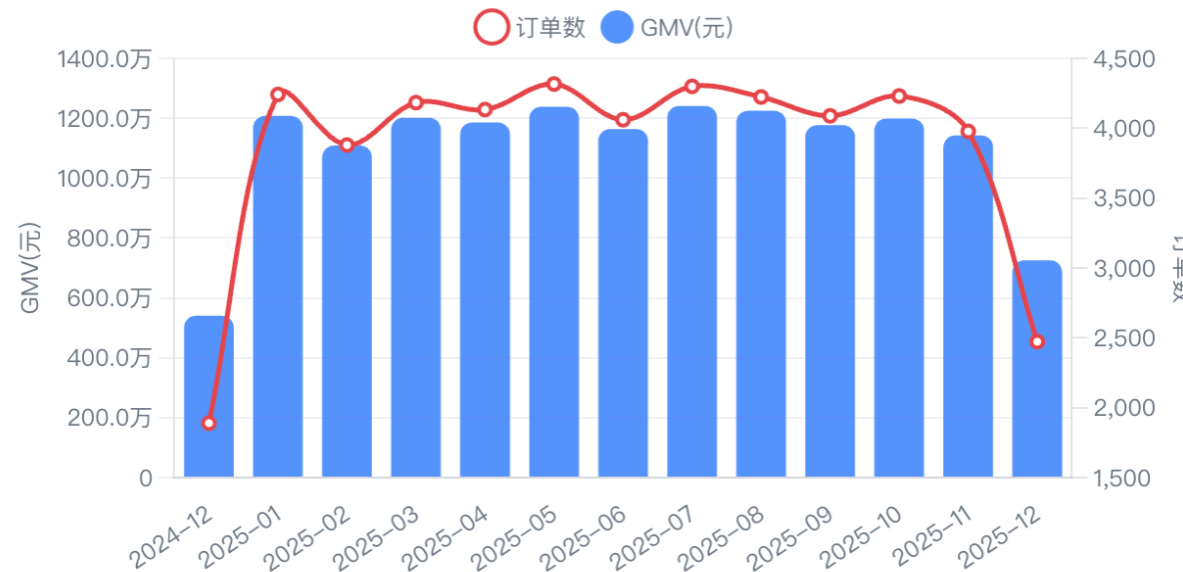
¥17.3 / ¥9,967.2

平均件数/单

2.99 件

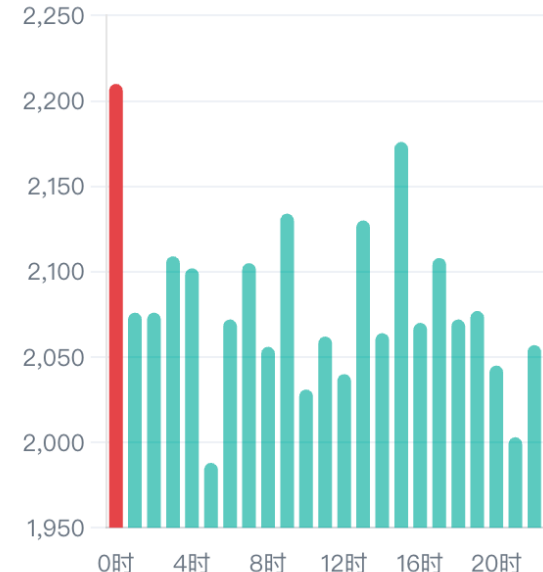
月度交易额与订单量趋势

柱：GMV (元) | 折线：订单数 (右轴)。2024-12 与 2025-12 为不完整月份



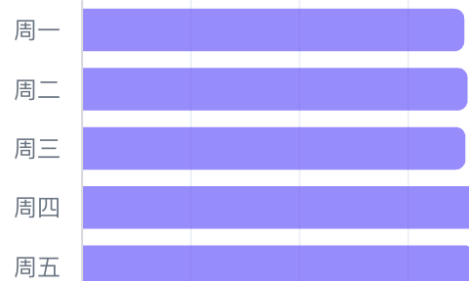
下单时段分布 (0-23 时)

全天订单在各时段的分布



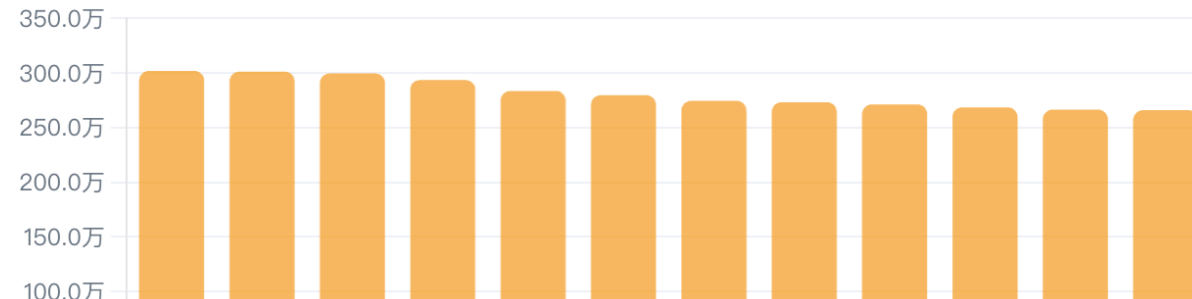
星期分布

周一至周日订单量



TOP 商品 · 交易额

按累计金额排名的前 12 个商品



04

总结与规划

总结与规划 从智能计算范式演进到 AI DLC 智能底座



2026 H2 围绕四大产品升级，持续与客户一起探索 AI 时代下数据作为核心资产的持续价值

总结：DLC → AI DLC 四大升级

已落地

01

计算范式升级

Ray × Spark

一份算力驱动数据处理、训练与推理

02

引擎能力升级

Xpark + Ray Libraries

多模态向量化引擎 + AI 全链路负载

03

元数据系统升级

TCLake

Data + AI 统一元数据与全流程血缘

04

Agentic 能力升级

MCP × Skills × CLI

平台支撑 Agent 进化 + 被 Agent 调用

2026 H2 规划：与客户共探数据核心资产价值

展望

01

产品持续演进

四大升级持续迭代，补齐能力、打磨体验，让平台更稳定、更高效。

02

客户共建验证

与灯塔客户一起在生产环境验证价值闭环，从案例到可复制打法。

03

数据核心资产

探索 AI 时代数据作为核心资产的持续价值创造与变现路径。

04

开放生态共建

Open Engine 战略，与开源生态共建共赢，让客户选最好用的引擎。

让数据真正成为 AI 时代的核心资产 —— 与客户一起持续进化。

感谢聆听



扫码了解更多